Python CPU Concurrency Cheatsheet

Concurrency = multiple tasks make progress over overlapping time periods.

```
├── Parallelism (execution property)
│   Parallelism = simultaneous execution on multiple processing units.
│
│   ├── Multiprocessing (single machine)
│   │   - Multiple OS processes
│   │   - True CPU parallelism (multi-core)
│   │   - Separate / isolated memory per process
│   │   - IPC for communication (overhead)
│   │   - Libraries:
│   │       • multiprocessing (stdlib)
│   │       • concurrent.futures.ProcessPoolExecutor (stdlib)
│   │       • joblib
│   │       • ray
│   │
│   └── Distributed systems (multiple machines)
│       - Multiple nodes / machines over a network
│       - Each node runs one or more processes
│       - Each process MAY use:
│           • multithreading
│           • async tasks (event loop)
│       - Parallelism across machines
│       - Network communication, partial failures
│       - Libraries:
│           • dask.distributed
│           • ray
│           • celery
│           • apache spark (pyspark)
│           • mpi4py
│
├── Concurrency mechanisms (how concurrency is implemented)
│
│   ├── Multithreading (CPython)
│   │   - Multiple threads in ONE process
│   │   - Threads SHARE process memory:
│   │       • heap, globals, objects, file handles
│   │       • each thread has its own stack & registers
│   │   - OS-scheduled (pre-emptive multitasking)
│   │   - Concurrent but NOT CPU-parallel for Python bytecode (GIL)
│   │   - CPU parallelism only via C extensions releasing the GIL
│   │   - Blocking I/O in one thread does NOT block others
│   │   - Requires locks/mutexes to avoid race conditions
│   │   - Libraries:
│   │       • threading (stdlib)
│   │       • concurrent.futures.ThreadPoolExecutor (stdlib)
│   │       • queue.Queue (stdlib, thread-safe)
│   │
│   ├── Multiprocessing
│   │   - Multiple OS processes (each may use threads)
│   │   - True CPU parallelism
│   │   - Memory NOT shared by default
│   │   - IPC for communication
│   │   - Higher startup & memory cost
│   │   - Bypasses Python GIL
│   │   - Libraries:
│   │       • multiprocessing (stdlib)
│   │       • concurrent.futures.ProcessPoolExecutor (stdlib)
│   │       • joblib
│   │       • pathos
│   │
│   └── Asynchronous / event-driven
│       - Single thread + event loop
│       - Many TASKS (not threads)
│       - Runtime switches tasks at `await` / I/O wait
│       - Cooperative concurrency
│       - No CPU parallelism
│       - One blocking call blocks the loop
│       - Very low overhead
│       - Libraries:
│           • asyncio (stdlib)
│           • aiohttp (async HTTP client/server)
│           • httpx (async HTTP client)
│           • trio
│           • curio
│           • uvloop (faster event loop)
│
├── Coordination & Synchronization
│   - Locks / mutexes
│   - Semaphores
│   - Condition variables
│   - Await / futures / promises
│   - Libraries:
│       • threading (Lock, RLock, Semaphore, Event, Condition)
│       • multiprocessing (Lock, Semaphore, Event, Barrier)
│       • asyncio (Lock, Semaphore, Event, Condition)
│       • concurrent.futures (Future)
│
└── Communication
    - Shared memory (threads)
    - IPC (processes)
    - Network (distributed systems)
    - Libraries:
        • queue.Queue (threads, stdlib)
        • multiprocessing.Queue (processes, stdlib)
        • multiprocessing.Pipe (processes, stdlib)
        • multiprocessing.Manager (shared state, stdlib)
        • redis / rabbitmq / kafka (message brokers)
        • zmq (pyzmq - distributed messaging)
```

Event Loop (Async)

```
┌──────────────┐
│  EVENT LOOP  │
│(Single Thread)│
└──────┬───────┘
       │ resumes
       ▼
┌──────────────┐
│ COROUTINE A  │
│  await I/O   │
└──────┬───────┘
       │ yields
       ▼
┌──────────────┐
│ COROUTINE B  │
│  await I/O   │
└──────┬───────┘
       │ yields
       ▼
┌──────────────┐
│ COROUTINE C  │
└──────────────┘
```

Multithreading

```
┌──────────────┐
│  ONE PROCESS │
│(Shared Memory)│
└──────┬───────┘
       │
   ┌───┴───┐
   │  GIL  │
   └───┬───┘
       │
   ┌───┴───┐
   │       │
┌──▼──┐   ┌──▼──┐
│Thread1│ │Thread2│
│ I/O  │  │ I/O  │
│ wait │  │ work │
└─────┘   └─────┘
(blocked) (running)
```

Multiprocessing

```
┌──────────────┐
│  PROCESS 1   │
│(Own Memory)  │
│  ┌────┐      │
│  │Core 1│    │
│  └────┘      │
└──────────────┘
      ‖ IPC
┌──────────────┐
│  PROCESS 2   │
│(Own Memory)  │
│  ┌────┐      │
│  │Core 2│    │
│  └────┘      │
└──────────────┘
```

CPU Core Usage

```
Multi-Core CPU:
┌────┬────┐
│Core 1│Core 2│
└────┴────┘
   │    │
┌──▼──┐ ┌▼────┐
│Proc 1│ │Proc2│
│ 100% │ │ 100%│
└────┘  └────┘
True Parallelism ✓

Single Thread/Async:
┌────┬────┐
│Core 1│Core 2│
└────┴────┘
   │  (unused)
┌──▼────┐
│1 Thread │
│  50%  │
└──────┘
```

Python rules of thumb:

Workload Type	Recommended Approach
CPU-bound work	`multiprocessing`, `joblib`, `ray`
Many I/O operations	`asyncio`, `aiohttp`, `httpx` (async) **OR** `threading`
Blocking libraries	`threading`, `concurrent.futures.ThreadPoolExecutor`
Large-scale data / ML	`dask`, `ray`, `spark`, `multiprocessing`
Web scraping	`asyncio` + `aiohttp` **OR** `threading` + `requests`
Task queues	`celery`, `rq`, `dramatiq`

Notes:

CPython: An interpreter that compiles Python source code into bytecode and then executes that bytecode on a virtual machine.

Coroutine: A function that can pause its execution and later resume from the same point, instead of always running from start to finish.